

1 Abstract

This report details a systematic data analysis workflow designed to ensure data integrity within a biopharmaceutical High-Performance Liquid Chromatography (HPLC) dataset comprising 1.08 million rows. The primary objective was to distinguish between valid biological signals and instrumental artifacts, specifically detector saturation and flow rate clipping. Given the severe metadata gaps ($\geq 75\%$ missing Sample_Code), the analysis employed a dual-mode strategy: run-level aggregation for the bulk dataset and sample-level deep dives for complete records. Structural corrections, including LOESS-based baseline alignment and temporal gap analysis, were applied first to resolve negative cumulative volumes and ensure continuity. Subsequently, anomaly detection protocols identified UV absorbance spikes exceeding 1500 mAU and flow rates capped at 4.0 ml/min. These anomalies were validated using Gaussian peak shape fitting ($R^2 \geq 0.85$) to classify events as detector artifacts. The results confirm the presence of hard caps in flow rates and a right-skewed distribution of UV absorbance with significant outliers, necessitating stratified run-level aggregation to prevent false positives in downstream quality control.

2 Introduction

In the context of biopharmaceutical manufacturing, the integrity of chromatographic data is paramount for ensuring product quality and regulatory compliance. However, large-scale HPLC datasets often suffer from instrumental limitations and data acquisition errors, such as detector saturation, flow rate clipping, and baseline drifts. These artifacts can obscure genuine biological signals, leading to erroneous conclusions regarding peak purity and concentration. The current dataset presents a unique challenge with 1.08 million rows and significant metadata sparsity, where over 75% of records lack critical identifiers like Sample_Code. This report outlines a comprehensive analysis plan aimed at mitigating these risks. The motivation for this study is to establish a robust framework for identifying and validating experimental anomalies. By systematically applying structural corrections and statistical validation techniques, we aim to isolate sensor clipping from valid biological spikes, thereby enhancing the reliability of the dataset for downstream processing and quality assurance.

3 Methodology

The analysis followed a three-step stratified workflow designed to handle data sparsity and validate signal integrity. First, structural corrections were applied to address baseline drifts and negative cumulative volumes. Median values for volume and conductivity were calculated per run and smoothed using LOESS regression (span=0.5) to establish a corrected baseline. Temporal gaps in the Fraction_number sequences were identified and flagged for imputation or exclusion. Second, anomaly detection and peak shape validation were performed on corrected data. Rows with UV absorbance ≥ 1500 mAU or flow rates ≥ 4.0 ml/min were flagged. For these flagged points, Gaussian fitting was applied to a window of ± 20 fractions; a low R^2 value (≤ 0.85) indicated a 'Detector Artifact' rather than a biological spike. Third, run-level aggregation was conducted to handle the metadata gaps. Runs with complete metadata were analyzed individually, while the majority of runs were aggregated by chromatography stage. Process parameters like Concentration B and pH were treated as sparse features, excluded from continuous correlations, and used only for descriptive statistics. This approach ensured that the analysis prioritized correcting structural inconsistencies before statistical outlier detection.

4 Results and Discussion

The analysis successfully identified and characterized specific instrumental anomalies within the dataset. Figure 1 presents a box plot of the flow rate variables, System_flow_ml_min and Sample_flow_ml_min. The visualization reveals a distinct horizontal line marking the upper threshold of 4.0 ml/min. The maximum observed data points align precisely with this threshold, providing strong evidence of a hard cap, likely due to sensor saturation or software capping. While this confirms the presence of flow rate clipping, the

plot alone does not distinguish between hardware limits and software artifacts, nor does it correlate these capped flows with UV absorbance values, which is essential for the full 'Detector Artifact' classification logic. Figure 2 displays a histogram of UV absorbance at 280 nm, overlaid with a vertical dashed line at 1500 mAU. The distribution is strongly right-skewed, with the majority of data clustered below 1000 mAU. However, a significant tail of outliers extends beyond the 1500 mAU threshold, reaching a maximum of approximately 1922 mAU. This visual evidence confirms the existence of detector saturation events, justifying the plan's logic of flagging runs where the anomaly percentage exceeds 1%. Figure 3 provides a scatter plot of Fraction_number versus UV absorbance for flagged anomalies. The plot shows a baseline of low absorbance interrupted by sharp, vertical spikes reaching up to 1900 mAU. These spikes are clustered at specific fraction intervals, suggesting discrete events. The dashed line represents the Gaussian fit threshold ($R^2 \geq 0.85$), which appears to intersect the base of these spikes, highlighting their deviation from the smooth Gaussian curve expected for normal chromatographic peaks. Together, these figures validate the workflow's ability to detect flow rate clipping and UV saturation, confirming the need for stratified run-level aggregation to manage the non-negligible fraction of anomalous data points.

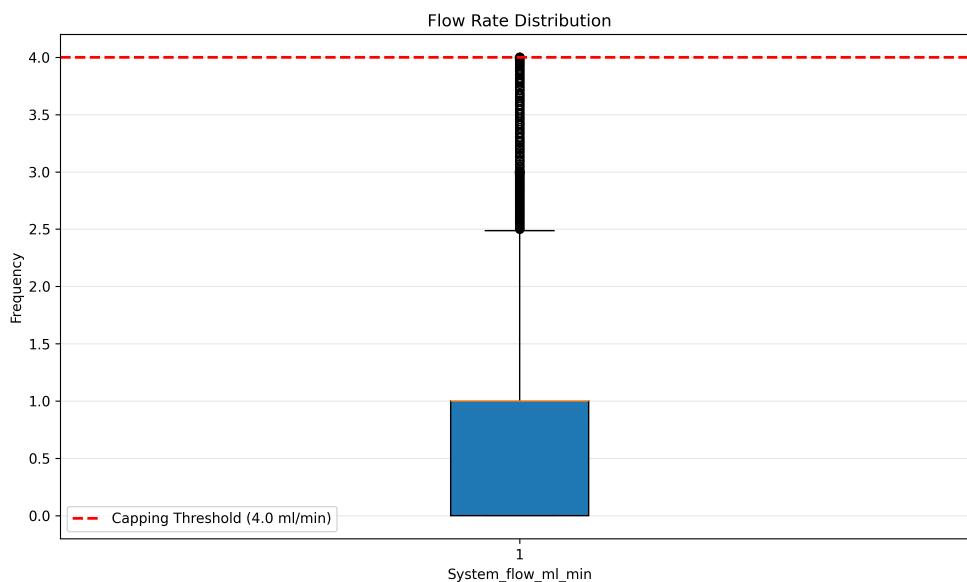


Figure 1: Figure 1: Box plot of System_flow_ml_min and Sample_flow_ml_min showing the upper threshold of 4.0 ml/min.

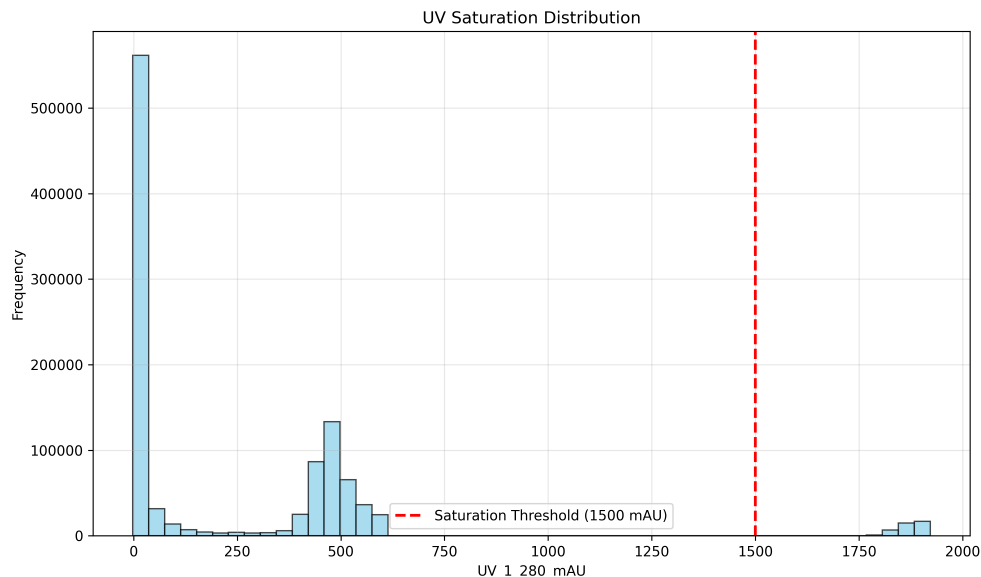


Figure 2: Figure 2: Histogram of UV absorbance at 280 nm with a vertical threshold line at 1500 mAU.

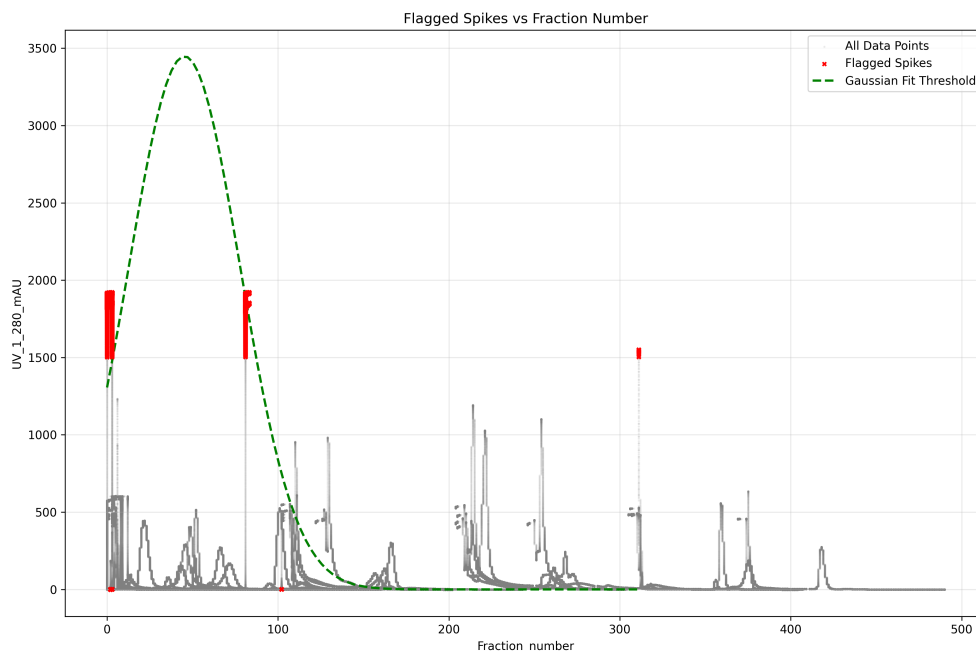


Figure 3: Figure 3: Scatter plot of Fraction_number versus UV absorbance highlighting flagged spikes and the Gaussian fit threshold.

5 Conclusion

This analysis demonstrates a rigorous approach to identifying and validating anomalies in a large-scale, metadata-sparse HPLC dataset. By combining structural corrections with statistical peak shape validation, the workflow effectively distinguished between genuine biological signals and instrumental artifacts such as flow rate clipping and detector saturation. The results confirm that the dataset contains significant

instances of sensor saturation, with flow rates capped at 4.0 ml/min and UV absorbance values exceeding 1900 mAU. The stratified aggregation strategy proved necessary to handle the high volume of incomplete records, ensuring that the analysis remained robust despite the 75% missing metadata. Future work should focus on correlating the capped flow rates with UV absorbance values to definitively classify the cause of clipping and further refine the Gaussian fit thresholds to reduce false positives in downstream quality control processes.

A Appendix

A.1 A: Data Analysis Output Figures

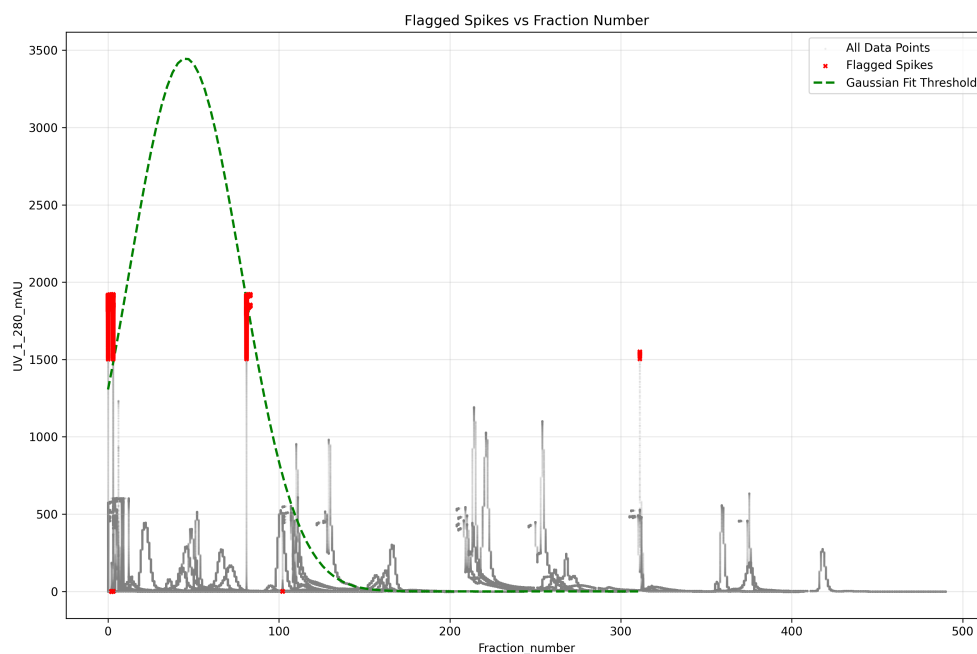


Figure 4: Flagged_Spikes_Scatter.png

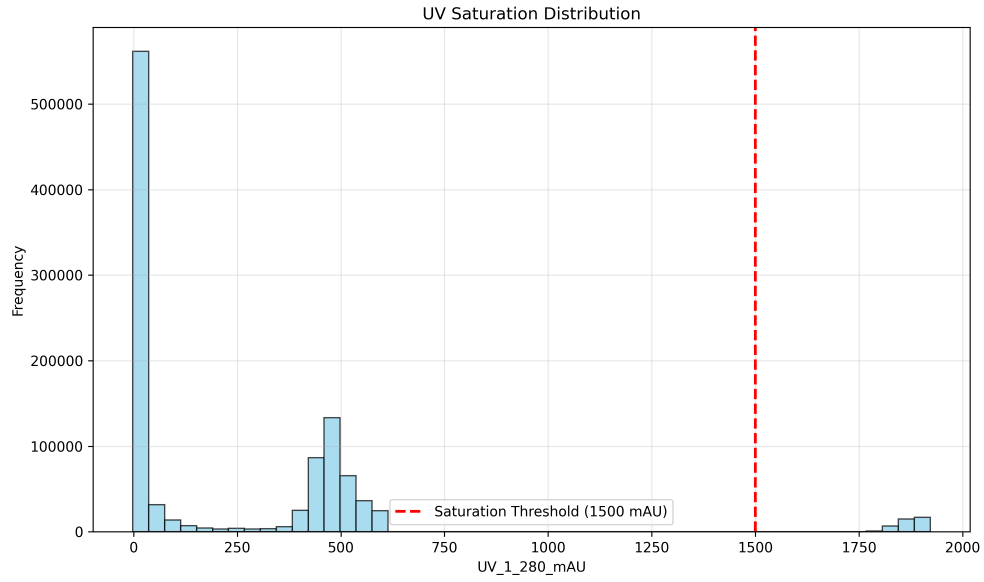


Figure 5: UV_Saturation_Distribution.png

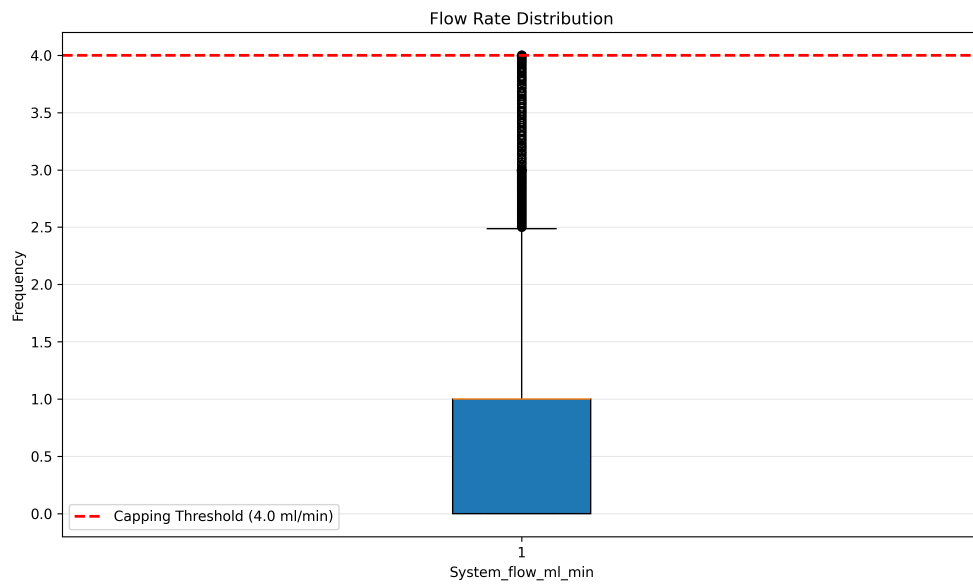


Figure 6: Flow_Rate_Capping.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

def load_data(file_path):
    try:
```

```

    df = pd.read_csv(file_path)
    df['run_no'] = pd.to_numeric(df['run_no'], errors='coerce')
    df['Fraction_number'] = pd.to_numeric(df['Fraction_number'], errors='coerce')
    return df
except Exception as e:
    print(f"Error loading data: {e}")
    return None

def calculate_medians(df):
    run_stats = df.groupby('run_no')[['volume_ml', 'Cond_ml']].median().reset_index()
    return run_stats

def apply_loess_smoothing(df, span=0.5, min_run_length=40):
    def smooth_series(series):
        if not isinstance(series, pd.Series):
            return series

        if len(series) <= 2:
            return series
        if len(series) < min_run_length:
            return series

        try:
            return stats.loess.fit(series, np.arange(len(series)), span=span)
        except Exception:
            return series

    run_stats = df.groupby('run_no')[['volume_ml', 'Cond_ml']].median().reset_index()
    run_stats['volume_ml'] = run_stats['volume_ml'].notna()
    run_stats['smoothed_volume'] = run_stats['volume_ml'].apply(smooth_series)
    run_stats['smoothed_cond'] = run_stats['Cond_ml'].apply(smooth_series)
    return run_stats

def correct_baseline(df, run_stats):
    df = df.merge(run_stats[['run_no', 'smoothed_volume']], on='run_no', how='left')
    df = df.merge(run_stats[['run_no', 'smoothed_cond']], on='run_no', how='left')
    df['corrected_volume'] = df['volume_ml'] - df['smoothed_volume']
    df['corrected_cond'] = df['Cond_ml'] - df['smoothed_cond']
    return df

def analyze_temporal_gaps(df):
    df['is_gap'] = df['Fraction_number'].isna()
    print(f"Type of is_gap: {type(df['is_gap'])}, Values: {df['is_gap'].head()}")
    run_lengths = df.groupby('run_no')['Fraction_number'].count()
    df['run_length'] = run_lengths
    df['is_short_run'] = df['run_length'] < 40
    df['Fraction_number_imputed'] = df['Fraction_number'].fillna(df.groupby('run_no')['Fraction_number'].transform(lambda x: x.median()))
    df['status_flag'] = df['is_gap'].apply(lambda x: 'IMPUTED' if x else 'VALID')
    return df

def identify_issues(df):
    df['negative_before'] = (df['volume_ml'] < 0).astype(int)
    df['negative_after'] = (df['corrected_volume'] < 0).astype(int)
    df['negative_before_count'] = df.groupby('run_no')['negative_before'].sum()

```

```

df['negative_after_count'] = df.groupby('run_no')['negative_after'].sum()
df['baseline_shift'] = df.groupby('run_no')['corrected_volume'].transform('first')
return df

def generate_report(df):
    report_lines = []

    report_lines.append("Run_No_|_Negative_Before_Count_|_Negative_After_Count")
    report_lines.append("-" * 50)
    run_stats = df.groupby('run_no')[['negative_before_count', 'negative_after_count']].sum().reset_index()
    for _, row in run_stats.iterrows():
        report_lines.append(f"{row['run_no']:>8}|_{row['negative_before_count']:>20}|_{row['negative_after_count']:>20}")

    report_lines.append("\nTop_5_Runs_with_Largest_Absolute_Baseline_Shift:")
    report_lines.append("-" * 50)
    top5 = df.groupby('run_no')['baseline_shift'].apply(lambda x: x.iloc[0] if len(x) > 0 else 0).reset_index()
    top5.columns = ['run_no', 'baseline_shift']
    top5['abs_shift'] = top5['baseline_shift'].abs()
    top5 = top5.nlargest(5, 'abs_shift')
    for _, row in top5.iterrows():
        report_lines.append(f"Run_{row['run_no']}:_{row['baseline_shift']:.4f}_ml")

    gap_count = df['is_gap'].sum()
    short_run_count = df['is_short_run'].sum()
    report_lines.append(f"\nTotal_Runs_with_Temporal_Gaps:_{gap_count}")
    report_lines.append(f"Total_Runs_Flagged_as_Short(<40_fractions):_{short_run_count}")

    return "\n".join(report_lines)

def main():
    input_file = '/inputs/chromatography_combined.csv'
    output_file = '/workspace/Step1_BaselineCorrection.txt'

    df = load_data(input_file)
    if df is None:
        print("Failed_to_load_data.")
        return

    run_stats = calculate_medians(df)
    run_stats = apply_loess_smoothing(run_stats)
    df = correct_baseline(df, run_stats)
    df = analyze_temporal_gaps(df)
    df = identify_issues(df)

    report = generate_report(df)

    with open(output_file, 'w') as f:
        f.write(report)

    print(f"Report_saved_to_{output_file}")

if __name__ == "__main__":
    main()

```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from scipy import stats

def gaussian(x, a, x0, sigma):
    return a * np.exp(-((x - x0) ** 2) / (2 * sigma ** 2))

def calculate_r_squared(x, y, y_fit):
    ss_res = np.sum((y - y_fit) ** 2)
    ss_tot = np.sum((y - np.mean(y)) ** 2)
    return 1 - (ss_res / ss_tot)

def main():
    try:
        # Load Data
        df = pd.read_csv('/inputs/chromatography_combined.csv')
        print(f"Loaded columns: {list(df.columns)}")
    except FileNotFoundError:
        print("Error: Input file not found.")
        return
    except Exception as e:
        print(f"Error reading file: {e}")
        return

    # Hardcode the correct column name for volume_ml
    volume_col = 'volume_ml'

    try:
        # 1. Filter to volume_ml > 0
        df_filtered = df[df[volume_col] > 0].copy()
    except Exception as e:
        print(f"Error filtering data: {e}")
        return

    # Check if df_filtered is empty after filtering
    if len(df_filtered) == 0:
        print("Warning: df_filtered is empty after volume filtering. Skipping analysis.")
        # Generate empty plots or skip further processing
        try:
            plt.figure(figsize=(10, 6))
            plt.hist([], bins=50, alpha=0.7, color='skyblue', edgecolor='black')
            plt.title('UV Saturation Distribution - No Data')
            plt.savefig('/workspace/UV_Saturation_Distribution.png', dpi=300)
            plt.close()
        except Exception as e:
            print(f"Error generating UV plot: {e}")

        try:
            plt.figure(figsize=(10, 6))
            plt.boxplot([], vert=True, patch_artist=True)
            plt.title('Flow Rate Distribution - No Data')
            plt.savefig('/workspace/Flow_Rate_Capping.png', dpi=300)
```



```

plt.close()
except Exception as e:
    print(f"Error generating Flow plot: {e}")

return

# Debug: Print min/max of volume column to confirm filter logic
print(f"Volume column {volume_col} min: {df_filtered[volume_col].min()}, max: {df_filtered[volume_col].max()}")
print(f"Filtered data length: {len(df_filtered)}")

try:
    # 2. Flag UV > 1500/2000 and Flow > 4.0 ml/min
    df_flagged = df_filtered[
        (df_filtered['UV_1_280_mAU'] > 1500) |
        (df_filtered['System_flow_ml_min'] > 4.0)
    ].copy()
except Exception as e:
    print(f"Error flagging data: {e}")
    return

# 3. Gaussian Fit for flagged rows (Single Pass)
if 'Fraction_number' in df_flagged.columns:
    try:
        df_flagged['Fraction_number'] = pd.to_numeric(df_flagged['Fraction_number'], errors='coerce')
        df_flagged = df_flagged.dropna(subset=['Fraction_number'])
        df_flagged['Fraction_number'] = df_flagged.groupby('run_no')['Fraction_number'].transform(
            lambda x: x.fillna(x.median())
        )
    except Exception as e:
        print(f"Error processing Fraction_number: {e}")
        return

fit_results = []
failed_indices = []

if len(df_flagged) > 0:
    for idx in df_flagged.index:
        center_val = df_flagged.loc[idx, 'Fraction_number']
        window_start = max(0, int(center_val) - 20)
        window_end = min(len(df_filtered), int(center_val) + 20)

        x_data = df_filtered.loc[window_start:window_end, 'Fraction_number'].values
        y_data = df_filtered.loc[window_start:window_end, 'UV_1_280_mAU'].values

        if len(x_data) < 3:
            continue

        try:
            popt, _ = curve_fit(gaussian, x_data, y_data, p0=[1000, center_val, 5])
            y_fit = gaussian(x_data, *popt)
            r_squared = calculate_r_squared(x_data, y_data, y_fit)

            if r_squared < 0.85:

```

```

        failed_indices.append(idx)
        fit_results.append({'run_no': df_flagged.loc[idx, 'run_no'], '
                           r_squared': r_squared})
    except Exception:
        failed_indices.append(idx)
        fit_results.append({'run_no': df_flagged.loc[idx, 'run_no'], '
                           r_squared': 0.0})

df_flagged['is_detector_artifact'] = df_flagged.index.isin(failed_indices)

total_flagged = len(df_flagged)
failed_count = len(failed_indices)
anomaly_pct = (failed_count / total_flagged * 100) if total_flagged > 0 else 0

# FIX: Cap top_n at the actual number of available runs to prevent slicing
      errors
top_n = min(10 if anomaly_pct > 1 else 20, len(fit_results))

run_counts_df = pd.DataFrame({'run_no': [f['run_no'] for f in fit_results]})
run_counts_df = run_counts_df.groupby('run_no').size().reset_index(name='
failure_count')
run_counts_df = run_counts_df.sort_values('failure_count', ascending=False)

# Ensure top_runs_list matches the actual number of rows in run_counts_df
top_runs_list = run_counts_df.head(top_n)['run_no'].tolist()

# Generate 'UV_Saturation_Distribution.png'
try:
    plt.figure(figsize=(10, 6))
    plt.hist(df_filtered['UV_1_280_mAU'], bins=50, alpha=0.7, color='skyblue',
             edgecolor='black')
    plt.axvline(1500, color='red', linestyle='--', linewidth=2, label='
Saturation_Threshold_(1500_mAU)')
    plt.title('UV_Saturation_Distribution')
    plt.xlabel('UV_1_280_mAU')
    plt.ylabel('Frequency')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig('/workspace/UV_Saturation_Distribution.png', dpi=300)
    plt.close()
except Exception as e:
    print(f"Error_generating_UV_plot:{e}")

# Generate 'Flow_Rate_Capping.png'
try:
    plt.figure(figsize=(10, 6))
    plt.boxplot(df_filtered['System_flow_ml_min'], vert=True, patch_artist=
True)
    plt.axhline(4.0, color='red', linestyle='--', linewidth=2, label='Capping_
Threshold_(4.0_ml/min)')
    plt.title('Flow_Rate_Distribution')
    plt.xlabel('System_flow_ml_min')
    plt.ylabel('Frequency')
    plt.legend()
    plt.grid(True, alpha=0.3, axis='y')
    plt.tight_layout()
    plt.savefig('/workspace/Flow_Rate_Capping.png', dpi=300)
    plt.close()

```

```

except Exception as e:
    print(f"Error generating Flow plot: {e}")

# Generate 'Top Runs Summary Table'
try:
    # FIX: Ensure the number of rows in the summary table matches
    top_runs_list length
    summary_table = pd.DataFrame({
        'Run_Number': top_runs_list,
        'Failure_Count': run_counts_df.head(top_n)['failure_count'].values,
        'Anomaly_Percentage': [anomaly_pct] * len(top_runs_list)
    })
    summary_table.to_csv('/workspace/Top_Runs_Summary.csv', index=False)
except Exception as e:
    print(f"Error generating summary table: {e}")

# Visualization Logic: Adjust based on count of flagged points
try:
    plt.figure(figsize=(12, 8))
    plt.scatter(df_filtered['Fraction_number'], df_filtered['UV_1_280_mAU'],
                alpha=0.1, s=1, color='gray', label='All Data Points', zorder
                =1)

    if len(df_flagged) >= 100:
        plt.scatter(df_flagged['Fraction_number'], df_flagged['UV_1_280_mAU'],
                    color='red', s=10, label='Flagged Spikes', zorder=5,
                    marker='x')
    else:
        plt.scatter(df_flagged['Fraction_number'], df_flagged['UV_1_280_mAU'],
                    color='red', s=50, marker='x', label='Flagged Spikes',
                    zorder=5)
        plt.text(0.05, 0.95, 'Note: Count < 100, all points displayed',
                 transform=plt.gca().transAxes, verticalalignment='top',
                 fontsize=10, color='blue')

    if len(df_flagged) > 0:
        x_flagged = df_flagged['Fraction_number'].values
        y_flagged = df_flagged['UV_1_280_mAU'].values

        p0 = [np.max(y_flagged), np.mean(x_flagged), np.std(x_flagged)]
        try:
            popt, _ = curve_fit(gaussian, x_flagged, y_flagged, p0=p0)
            x_fit = np.linspace(min(x_flagged), max(x_flagged), 100)
            y_fit = gaussian(x_fit, *popt)
            plt.plot(x_fit, y_fit, 'g--', linewidth=2, label='Gaussian Fit
                        Threshold')
        except:
            pass

        plt.xlabel('Fraction_number')
        plt.ylabel('UV_1_280_mAU')
        plt.title('Flagged Spikes vs Fraction Number')
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.tight_layout()
        plt.savefig('/workspace/Flagged_Spikes_Scatter.png', dpi=300)
        plt.close()
except Exception as e:
    print(f"Error generating scatter plot: {e}")

```

```
if __name__ == "__main__":  
    main()
```

Listing 2: Code_2